

Threat Model and Security Implications of an OpenClaw Agent on EC2 with Read-Only Gmail Access

Scope and assumptions

This document analyses a specific high-risk configuration: an OpenClaw agent running on a cloud virtual machine (an “EC2 instance”) with network egress to the public internet and OAuth-granted **read-only** access to a user’s Gmail mailbox. The analysis is written from a defensive threat-modelling angle: what could go wrong, why “read-only” is still dangerous, what prerequisites each abuse case requires, and how to reduce blast radius.

Two assumptions from your memo are treated as *design constraints* rather than edge cases:

- The agent can be coerced via **prompt injection** (including indirect prompt injection sourced from content the agent reads, such as emails), and current mitigations reduce but do not eliminate this risk. OpenClaw’s own security guidance explicitly states that prompt injection “is not solved” and that system prompts are “soft guidance only”; hard controls must come from tool policy, approvals, sandboxing, and allowlists. ¹
- The Gmail permission is genuinely **read-only**: the agent cannot send mail, delete mail, change settings, or configure forwarding rules through Gmail APIs. This corresponds to the Gmail API OAuth scope that provides “Read all resources and their metadata—no write operations.” ²

It is crucial to define what “read-only Gmail access” means in practice. In the Gmail API documentation, the `gmail.readonly` scope is categorised as **Restricted** and is broad in read capability: it can read Gmail resources and their metadata while disallowing write operations. ² Restricted scopes are treated as highly sensitive in Google’s ecosystem; the same documentation notes that if restricted-scope data is stored on servers or transmitted, additional verification and potentially a security assessment are required—an implicit signal that “read-only” does not mean “low risk.” ²

Finally, this deployment is “agentic”: OpenClaw is designed to integrate with external APIs and tools and can be configured with high-risk tools (e.g., execution and web capabilities). Its own docs recommend limiting tools like `exec`, browser control, and web fetching to trusted agents or explicit allowlists precisely because these tools convert untrusted text into real-world actions. ³

System model and attack surface

A simplified system model for the scenario is:

1. **Inbound content sources:** email bodies, attachments, links inside emails, and any other retrieved content the agent is tasked to process (web pages, pasted text, documents). OpenClaw explicitly warns that prompt injection can occur through any untrusted content the bot reads, including “emails, docs, attachments.” ¹

2. **Agent reasoning layer:** an LLM-driven agent that “blurs the line” between data and instructions. This is the core risk highlighted in indirect prompt injection research: attackers can embed instructions in content likely to be retrieved and processed by an LLM-integrated application. ⁴
3. **Tools and outputs:** capabilities that can transform the agent’s interpretation into actions—network calls, retrieval, file reads/writes, process execution, scheduled tasks, and interactions with external services. OpenClaw’s security guidance repeatedly emphasises that **hard enforcement** must be implemented at the tool layer (policy, approvals, sandboxing), not merely in prompts. ¹

There are three primary attacker entry points relevant to your memo:

Indirect prompt injection via email content

If the agent ingests email text into its prompt context (for summarisation, classification, “inbox cleanup,” or workflow automation), then an attacker can inject adversarial instructions by sending a crafted email. This is a canonical “indirect prompt injection” pattern: the attacker does not need direct access to the bot’s chat interface; they only need to control content the bot reads. ⁵

Prompt injection via links/attachments the agent follows

OpenClaw recommends treating “links, attachments, and pasted instructions as hostile by default,” reflecting the same risk: content can carry instructions that attempt to trigger tool calls or exfiltrate context. ¹

Direct access via misconfiguration (secondary but common in practice)

Even if your scenario is “email-only injection,” it matters that OpenClaw is often deployed as an always-on agent connected to messaging surfaces. The OpenClaw repository warns to treat inbound DMs as untrusted input and defaults to a DM pairing/allowlist approach (unknown senders do not get processed without approval). ⁶ The practical reason for these defaults is that exposed agents can be treated as “AI backdoor” surfaces; industry writeups highlight both direct and indirect instruction attacks via emails and web pages. ⁷

image_group{"layout":"carousel","aspect_ratio":"16:9","query":["LLM agent architecture diagram tool calling prompt injection","AWS EC2 instance metadata service diagram 169.254.169.254","OAuth token flow diagram Gmail API read-only scope","OpenClaw AI agent security sandboxing diagram"],"num_per_query":1}

Why read-only Gmail access is still high impact

The central conclusion of your memo—that “read-only” still implies severe compromise—is consistent with well-established attacker tradecraft and modern LLM-agent threat models.

The mailbox is a high-value dataset even without write permissions

Email collection is explicitly recognised as a common adversary technique because emails routinely contain sensitive information (personal data, trade secrets, authentication workflows, and context useful for follow-on operations). ⁸ Read-only access is enough to run broad or targeted collection, including searching and extracting specific categories of messages (finance, identity, security alerts, invoices, internal projects).

This matters because email is not just “messages”—it is often the **hub** of a person’s digital identity. A security strategy article about personal email compromise characterises personal email as a “treasure trove” that includes “password reset links,” and notes that compromise can lead to reputational risk and can be leveraged toward further credential compromise. ⁹

Read-only email enables downstream account takeover patterns

Even when an agent cannot *send* email, reading email can enable account takeovers elsewhere in at least three common ways:

- **Magic links / one-time login links:** Many services authenticate by sending login links to email. Read access can be enough to capture and use these links before the user notices. Defensive guidance in multiple frameworks emphasises using out-of-band mechanisms to verify critical actions initiated via email (password resets, transactions), which implicitly acknowledges that email visibility is a powerful capability for attackers. ¹⁰
- **Password reset workflows:** Good practice guidance for service operators recommends one-time reset links (rather than emailing passwords), but the flip side is that anyone who can read the mailbox can often capture those one-time links. ¹¹
- **Email-delivered verification codes:** Many services still use email as a verification step. If an attacker can read those codes, MFA can be partially neutralised for accounts that rely on email as a second factor (or as a fallback). This is directly reflected in MITRE’s mitigation language recommending out-of-band verification for email-initiated critical actions. ¹⁰

Prompt injection turns the mailbox into an instruction channel with “hands”

The shift introduced by an agent like OpenClaw is not only that the mailbox can be read—it is that mailbox contents can become an **execution driver**.

OpenClaw’s security doc is unusually direct: prompt injection remains unsolved; system prompts are not a security boundary; and prompt injection can happen through “emails” and other retrieved content, even if only trusted users can message the bot. ¹ Academic and industry work on indirect prompt injection formalises the same hazard: LLM-integrated applications blur the line between “data” and “instructions,” and adversaries can exploit this to cause unintended actions, data theft, or tool/API misuse. ¹²

Practical attack paths in the OpenClaw + read-only Gmail scenario

This section enumerates the most relevant abuse cases, focusing on your memo’s key question: **what can be done with read access** once the agent is coerced (e.g., by a malicious email).

Mailbox-scale exfiltration

What happens: The agent is tricked into exporting mailbox contents (full messages, snippets, attachments, or a filtered subset) to an attacker-controlled destination. “Read-only” does not prevent copying; it only prevents modifying the mailbox.

Why it is realistic: Exfiltration over web services is a standard adversary technique: attackers often use legitimate external web services or ordinary HTTPS traffic patterns to move data out, because those channels blend in with normal network activity. ¹³ With an agent that has web access and tool

autonomy, “exfiltration” becomes a plausible consequence of a single successful instruction hijack. OpenClaw explicitly calls out “exfiltrating context or triggering tool calls” as a typical prompt injection risk when tools are enabled. ¹

What the attacker needs (capabilities):

The attacker needs (a) a way to get content into the agent’s context (e.g., sending an email that will be processed), and (b) a mechanism to cause the agent to make outbound network requests or interact with external services. This is exactly why OpenClaw recommends limiting high-risk tools like `web_fetch`, `web_search`, `browser`, and `exec`. ¹

Targeted secret harvesting from email

What happens: Instead of dumping everything, the agent searches for high-value content: API keys, password reset messages, invoices, identity documents, crypto exchange notifications, payroll files, or sensitive internal threads. The output may be smaller but far more damaging.

Why it is realistic: Email is often a repository of “secrets in plain sight,” and MITRE explicitly notes that emails may contain sensitive data valuable to adversaries. ⁸ In an agent context, search-and-extract tasks are easy to describe in natural language, and indirect prompt injection research highlights “data theft” as a core impact category. ¹⁴

What the attacker needs:

Read-only mailbox access (already granted), plus the ability to steer the agent to run searches and export results. This steering can come from indirect prompt injection embedded in email text. ¹⁵

Account takeover chaining via email-only authentication and resets

What happens: The agent is pushed into a workflow that escalates from “reading email” to “owning other accounts” by leveraging email-based authentication flows (reset links, magic links, email OTPs).

Why it is realistic: Security guidance aimed at protecting personal email explicitly emphasises that personal email contains password reset links and that personal email compromise can be used to pursue broader credential compromise. ⁹ MITRE’s guidance to verify email-initiated critical actions out-of-band further underscores how often email is the pivot point for real compromise. ¹⁰

What the attacker needs:

In addition to mailbox read access, the agent needs some ability to interact with external websites (e.g., using a browser tool, or API clients) to complete the downstream login/reset steps. OpenClaw treats browser control as especially sensitive because a browser profile can carry authenticated sessions and thus “equivalent to operator access” to whatever that profile can reach. ¹

High-confidence impersonation and social engineering

What happens: The attacker uses mailbox visibility to craft convincing fraud, even if they cannot send from the compromised Gmail account. They can learn relationships, vendor/customer context, writing style, ongoing negotiations, travel plans, and internal processes. They can then impersonate the victim via other channels (lookalike domains, other mailbox compromises, messaging apps), or target third parties with highly credible pretexts.

Why it is realistic: Business email compromise is fundamentally a trust-exploitation problem. Government and law enforcement messaging describes BEC scams as evolving and financially damaging, targeting both business and personal transactions and increasing in exposed losses. ¹⁶ Email visibility is a force multiplier for these scams because it provides ground truth about who to target, what to ask for, and when.

What the attacker needs:

Only read access and time. Even without sending ability, the mailbox provides the intelligence needed for “confidence” fraud and spearphishing.

Persistence as an “AI backdoor” via agent memory and automation

What happens: The attacker establishes durable influence over the agent so that compromise is not a one-off. In agent frameworks, persistence can include scheduled automations, durable configuration state, or “memory” that continues across sessions.

Why it is realistic: OpenClaw is built to be always-on and supports long-running behaviour (e.g., daemonised gateway, persistent sessions, cron-like capabilities). ¹⁷ Industry analysis of OpenClaw highlights that it may store configuration and interaction history locally, enabling behaviour persistence across sessions—useful for productivity, but also for persistence if an attacker gains influence. ¹⁸

What the attacker needs:

A single successful injection that causes the agent to store attacker-chosen state or set up recurring behaviours, plus continued access to untrusted content channels (e.g., the attacker can keep sending “instruction emails”).

EC2-specific escalation risks when the agent is cloud-hosted

Running the agent on `Entity["company","Amazon Web Services","cloud provider"]` changes the blast radius in two meaningful ways: (1) the host becomes a cloud workload that may have cloud-native credentials and metadata access, and (2) outbound egress becomes easier to operationalise at scale.

Instance Metadata Service and cloud credential theft

The **Instance Metadata Service (IMDS)** is reachable from within an instance and is used to access instance metadata; AWS documents that IMDS supports different versions (IMDSv1, IMDSv2) and that IMDSv2 requires session tokens. ¹⁹

From a threat-modelling standpoint: if the OpenClaw process (or its tool sandbox) can access IMDS, and the instance has an attached IAM role, then IMDS becomes a high-value target because it can expose credentials and other sensitive instance context.

This is not speculative: MITRE ATT&CK describes “Cloud Instance Metadata API” access as a credential access technique; it notes that metadata APIs may include sensitive data such as credentials and user data scripts, and that the metadata API is typically hosted at `169.254.169.254`. ²⁰

Why this matters in your scenario: prompt injection provides an attacker a way to cause the agent to make “local” HTTP calls it would not otherwise make. Once cloud credentials are obtained, the attacker may pivot to other cloud resources (object storage, secrets, logs, snapshots), depending on role permissions.

Token and secret handling on the EC2 filesystem

Even if Gmail access is limited by OAuth scope, the **OAuth tokens themselves** (refresh/access tokens) become sensitive assets. Google's OAuth best practices emphasise that user tokens must be stored securely at rest, never transmitted in plain text, and revoked when no longer needed. ²¹

OpenClaw's security guidance aligns with this: it recommends keeping "secrets out of the agent's reachable filesystem" and running sensitive tool execution in a sandbox. ¹

In other words, the risk is not just "email content exfiltration," but also "credential material exfiltration" (Gmail tokens, model provider keys, gateway auth tokens, logs), which could enable continued or expanded access.

Supply chain and "skills" as an execution path

Your memo frames the threat mainly as prompt injection, but in real deployments OpenClaw's extensibility matters: third-party "skills" are executable code, and multiple recent security writeups warn that skills can be a malware distribution vector.

A recent report on malicious OpenClaw skills describes skills as "folders of executable code" that can interact with the filesystem and network once installed and enabled, and notes the ecosystem has already been targeted for malware distribution. ²² This is relevant to the EC2 scenario because cloud-hosted agents often run unattended; if an attacker can influence "install/update" actions (through injection or operator error), the instance may be converted into a more traditional persistent foothold.

Risk rating and prioritised scenarios

This section proposes a pragmatic rating model (not a formal standard): **Likelihood** (1–5) × **Impact** (1–5) = **Risk**. Likelihood assumes the adversary can send email to the victim and that the agent processes untrusted emails (as in your memo). Impact assumes the mailbox contains typical modern digital identity artefacts (subscriptions, invoices, password resets).

Mailbox exfiltration at scale

Likelihood: 5 / Impact: 5 / Risk: Critical

Read-only Gmail access is sufficient to collect sensitive information (a documented adversary objective in email collection). ⁸ With an agent that can make outbound web requests, exfiltration over web services is a realistic data-loss channel. ²³

Targeted "secret mining" → downstream compromise

Likelihood: 4 / Impact: 5 / Risk: Critical

Email commonly contains password reset links and security notifications, and compromise can be leveraged toward broader credential compromise. ⁹ Indirect prompt injection research explicitly includes data theft and tool misuse as practical impacts. ²⁴

Account takeover chaining via email-based logins/resets

Likelihood: 4 / Impact: 5 / Risk: Critical

Even well-designed reset systems rely on email as a delivery channel for one-time links; service-

operator guidance favours one-time reset links over emailing passwords. ²⁵ If an attacker can read the mailbox, they can often pivot into other accounts that use email for recovery or authentication, which is why defender guidance stresses out-of-band verification for email-initiated critical actions. ²⁶

EC2 credential pivot via Instance Metadata Service

Likelihood: 3 / Impact: 5 / Risk: High (can be Critical)

IMDS is reachable from inside the instance and has distinct security considerations (IMDSv2 token requirement, configuration options). ¹⁹ MITRE explicitly documents metadata API access as a way to collect credentials and other sensitive data from cloud instances. ²⁰ The impact depends heavily on how permissive the instance role is.

Social-engineering amplification and BEC-style fraud enablement

Likelihood: 3 / Impact: 5 / Risk: High

Even without sending capabilities, mailbox visibility enables accurate impersonation pretexts. Government and law enforcement communications emphasise BEC's scale and evolution, including increases in exposed losses and ongoing targeting of both business and personal transactions. ¹⁶

Persistence through automation/memory

Likelihood: 3 / Impact: 4 / Risk: High

OpenClaw is designed to be always-on, supports automation features, and its security guidance discusses compromise response in a way that assumes tokens can leak or unexpected tool calls can occur. ¹⁷ If the attacker can influence persistent state, the harm becomes recurring rather than a single incident.

Defensive controls for OpenClaw-on-EC2 with Gmail read-only

The key design principle from your memo is correct: you cannot rely on "read-only Gmail" or "a strong system prompt" as the primary safety mechanism. You must design so that **words do not directly become actions**—especially actions that move data off the box or expand privilege.

Reduce Gmail exposure below "read-only" where possible

If the agent does not truly need message bodies or attachments, prefer narrower Gmail scopes. The Gmail scope list includes a metadata-only scope (`gmail.metadata`) that reads headers and metadata but not message bodies or attachments, which can materially reduce what an attacker can steal through the Gmail integration. ²

When message bodies are necessary, apply *application-layer minimisation*: only fetch the subset of messages required for a task, redact or summarise aggressively, and avoid storing raw mail content on disk. This aligns with the fact that restricted Gmail scopes are treated as high sensitivity in Google's access model. ²

Split the agent into a "reader" and an "actor"

OpenClaw explicitly recommends using a **read-only or tool-disabled reader agent** to process untrusted content, then passing only a summary into your tool-enabled agent. ¹

In this scenario, that means:

- A **mail-reader** agent: can fetch emails, but has *no* web browsing, no exec, no file access beyond a scratch area, and no ability to call arbitrary external endpoints.
- A **tool-enabled** agent: can take structured, sanitised tasks, but does **not** have direct access to raw mailbox content.

This does not “solve” injection, but it breaks the most dangerous chain: untrusted text → privileged tools.

Constrain high-risk tools and require approvals for data movement

OpenClaw’s own hardening guidance is directly applicable: limit high-risk tools (`exec`, `web_fetch`, `web_search`) to trusted agents or explicit allowlists, and treat system-prompt rules as insufficient without tool policy enforcement. ¹

For this specific threat model, the most important approval gates are:

- Any action that sends data off-host (uploads, webhooks, posting to external services). This maps to known exfiltration techniques where web services are used as an exfil channel. ¹³
- Any action that accesses credential stores or sensitive local paths (where OAuth tokens, config, and logs might live). This aligns with Google’s requirement that tokens be stored securely and with OpenClaw’s emphasis on keeping secrets out of reachable filesystems. ²⁷

Enforce sandboxing and isolate secrets from the agent runtime

OpenClaw recommends running sensitive tool execution in a sandbox and keeping secrets out of the agent’s reachable filesystem, noting that sandboxing is opt-in and must be configured correctly. ¹

From a cloud deployment standpoint, this typically means:

- Run the gateway and tools with strict OS-level isolation (containers/VM isolation boundaries where feasible).
- Ensure the sandbox cannot read the host’s secret material (tokens, SSH keys, cloud credentials).
- Treat logs and transcripts as sensitive artefacts because they can contain tool arguments and data the model saw. ¹

Lock down network egress instead of assuming “internet access is harmless”

Your memo correctly identifies that the ability to make arbitrary internet calls is what converts read access into exfiltration capability. Exfiltration over common web channels is a well-documented technique precisely because it blends into normal traffic. ¹³

Defensive implication: if the agent only needs to talk to Gmail endpoints and a small number of model/provider endpoints, restrict outbound traffic to those destinations (via egress proxy, firewall policy, or other network controls). This is one of the few controls that remains effective even if the agent is instruction-hijacked.

Harden or disable EC2 instance metadata access

AWS documents that IMDSv2 requires session tokens and provides configuration options around metadata retrieval. ¹⁹ MITRE frames metadata API access as a credential access technique and notes that metadata APIs may expose credentials and user data scripts. ²⁰

For this threat model:

- If the instance does not need an instance role, do not attach one (eliminates the “free cloud credentials” target).
- If IMDS is not needed, disable it; otherwise require IMDSv2 and restrict access to IMDS so only explicitly authorised processes can reach it.

Prepare incident response for “mailbox read-only token compromise”

Because OAuth tokens can be stolen and abused, Google’s best practice guidance emphasises token revocation and secure storage. ²¹ OpenClaw’s own incident response checklist includes rotating gateway auth, revoking/rotating model provider credentials, and reviewing logs/transcripts for unexpected tool calls. ¹

A practical IR runbook for this specific scenario should include:

- Immediate Gmail OAuth token revocation and re-authorisation with minimal scopes. ²⁸
- Review of mailbox access patterns and identification of sensitive email categories at risk (password resets, banking, contracts). ²⁹
- Reset of passwords on high-value external accounts that rely on email recovery, using out-of-band verification where possible. ²⁶

Appendix of additional abuse cases and adjacent risks

Below are additional ways “read-only Gmail + agent tools” can be abused, phrased as threat hypotheses (not step-by-step exploits). Each assumes the attacker can inject instructions through content the agent reads (e.g., email), consistent with indirect prompt injection models. ⁵

- **Selective exfiltration to blend in:** exfiltrate only “interesting” messages (finance, legal, security alerts) to avoid detection while maximising impact. ³⁰
- **Conversation-history leakage:** coerce the agent to disclose or export its own logs/transcripts, which can contain secrets, tool arguments, and sensitive context. OpenClaw warns that verbose outputs can expose tool outputs and that logs/transcripts should be reviewed during incidents. ¹
- **Credential replay against other services:** mail-derived intelligence enables highly targeted phishing without needing to send from the compromised Gmail account; this is a common pattern in BEC-style fraud. ¹⁶
- **Identity/relationship mapping:** build a high-fidelity graph of contacts, trusted vendors, and internal processes from email threads—useful for fraud and extortion. Email collection is recognised as valuable for sensitive info and operational insight. ⁸
- **Document/attachment “Trojan text”:** hide adversarial instructions inside attachments or long email threads that the agent is tasked to summarise; this is a core indirect prompt injection mechanism (“hide instructions in the data sources the system uses”). ³¹
- **Forced browsing to an attacker-controlled instruction page:** drive the agent to consult a specific site repeatedly, turning the site into a de facto command channel. This maps to well-

known command-and-control concepts where web protocols are used to communicate with compromised systems. ³²

- **Pivot into cloud resources via metadata and roles:** if the EC2 instance has an attached role, metadata access can expose credentials, enabling lateral movement into other cloud services. ³³
- **“Skill” or plugin abuse:** trick operators (or the agent, if permitted) into installing untrusted skills; recent reporting describes skills as executable code with filesystem and network access and notes malware distribution attempts. ³⁴
- **Extortion via private information exposure:** attackers don’t need write access to cause harm; reading sensitive material can be sufficient for coercion, harassment, doxxing, or reputational damage. Email compromise guidance highlights reputational and organisational risk from personal email compromise. ³⁵
- **Browser-driven compromise analogies:** in adjacent security domains, browser exploitation frameworks exist that provide a command-and-control style interface over compromised clients; while not identical, the conceptual lesson is transferable—once a “client” executes attacker-directed actions and can beacon out, the attacker can manage it like an agent. ³⁶

1 3 5 15 Security - OpenClaw

<https://docs.openclaw.ai/gateway/security>

2 Choose Gmail API scopes | Google for Developers

<https://developers.google.com/workspace/gmail/api/auth/scopes>

4 12 14 24 Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection

https://arxiv.org/abs/2302.12173?utm_source=chatgpt.com

6 17 GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way.

<https://github.com/openclaw/openclaw>

7 18 What Security Teams Need to Know About OpenClaw, the AI Super Agent

<https://www.crowdstrike.com/en-us/blog/what-security-teams-need-to-know-about-openclaw-ai-super-agent/>

8 29 30 Email Collection, Technique T1114 - Enterprise

https://attack.mitre.org/techniques/T1114/?utm_source=chatgpt.com

9 35 Securing the corner office at home: How CEOs can secure their personal email | Google Cloud Blog

<https://cloud.google.com/transform/how-ceos-can-secure-their-personal-email>

10 26 Remote Email Collection, Sub-technique T1114.002

https://attack.mitre.org/techniques/T1114/002/?utm_source=chatgpt.com

11 25 Passwords in online services | ICO

https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/security/a-guide-to-data-security/passwords-in-online-services/?utm_source=chatgpt.com

13 23 Exfiltration Over Web Service, Technique T1567 - Enterprise

https://attack.mitre.org/techniques/T1567/?utm_source=chatgpt.com

16 Business Email Compromise: The \$55 Billion Scam

https://www.ic3.gov/PSA/2024/PSA240911?utm_source=chatgpt.com

19 Access instance metadata for an EC2 instance

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instancedata-data-retrieval.html?utm_source=chatgpt.com

20 33 Unsecured Credentials: Cloud Instance Metadata API, Sub-technique T1552.005 - Enterprise | MITRE ATT&CK®

<https://attack.mitre.org/techniques/T1552/005/>

21 27 28 Best Practices | Authorization Resources | Google for Developers

<https://developers.google.com/identity/protocols/oauth2/resources/best-practices>

22 34 Malicious OpenClaw 'skill' targets crypto users on ClawHub — 14 malicious skills were uploaded to ClawHub last month | Tom's Hardware

<https://www.tomshardware.com/tech-industry/cyber-security/malicious-moltbot-skill-targets-crypto-users-on-clawhub>

31 Indirect Prompt Injection: Generative AI's Greatest Security Flaw

https://cetas.turing.ac.uk/publications/indirect-prompt-injection-generative-ais-greatest-security-flaw?utm_source=chatgpt.com

32 Command and Control, Tactic TA0011 - Enterprise

https://attack.mitre.org/tactics/TA0011/?utm_source=chatgpt.com

36 Testing for Stored Cross Site Scripting

https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/07-Input_Validation_Testing/02-Testing_for_Stored_Cross_Site_Scripting?utm_source=chatgpt.com